Understanding and Enhancing DiskANN: Towards Adaptive and Information-Driven Graph-Based ANN Systems

Abhijeet Kumar (DA24B001)
Krish Dange (DA24B011)
Sai Jagadeesh (DA24B013)

April 5, 2026

Abstract

Approximate Nearest Neighbor (ANN) search is a fundamental problem in large-scale machine learning systems, where achieving high recall with low latency is critical. DiskANN, which utilizes the Vamana graph construction algorithm, offers an efficient way to conduct ANN search by leveraging graph-based indexing and SSD optimization.

In this paper, we explore the design of the DiskANN framework as well as its key merits and drawbacks in terms of balancing accuracy, memory, and latency requirements. We highlight several aspects related to DiskANN that need improvement, namely the static nature of its graph construction, the use of distance-related heuristics, and a lack of adaptivity. To that end, we propose directions of research that could address these issues.

Introduction

Nearest Neighbor Search lies at the heart of a wide range of applications, including recommendation systems, information retrieval, and computer vision problems, where input data points reside in high-dimensional vector spaces. Due to the curse of dimensionality, performing exact nearest neighbor search becomes computationally expensive. As a result, Approximate Nearest Neighbor (ANN) search is commonly used as a practical alternative.

Graph-based techniques represent one of the most advanced approaches for implementing ANN due to their efficient navigational properties. DiskANN employs this technique to construct an optimized graph structure that enables fast and high-recall search at a large scale.

DiskANN: Functionality and Workflow

DiskANN is a graph-based Approximate Nearest Neighbor Search (ANNS) system designed to scale to billion-point datasets while operating under limited main memory. It combines a navigable graph structure with SSD-based storage and vector compression to achieve high recall and low query latency [**?**]. The dataset is modeled as a set of points

$$P = \{p_1, \ldots, p_n\}, \quad x_p \in R^d,$$

with similarity measured using Euclidean distance

$$d(p, q) = \|x_p - x_q\|_2.$$

The goal is to retrieve k approximate nearest neighbors efficiently, with quality measured by recall@k:

$$recall@k = \frac{|X \cap G|}{k}.$$

Graph Construction (Vamana).

DiskANN builds a directed graph $G = (P, E)$ where each node has at most R outgoing edges. The graph is initialized randomly and refined iteratively. For each point p, a greedy search is performed from a fixed start node s to obtain a candidate set V_p. A pruning step then selects a subset of neighbors using the α-RNG condition:

$$\alpha \cdot d(p^*, p') \leq d(p, p') \;\Rightarrow\; p' is removed,$$

ensuring that only diverse and informative edges are retained. This results in a bounded-degree graph:

$$|N_{out}(p)| \leq R.$$

A key property of Vamana is that it encourages multiplicative progress toward the query:

$$d_{next}(q) \leq \frac{1}{\alpha} \cdot d_{current}(q), \quad \alpha > 1,$$

which reduces the number of traversal steps and improves search efficiency.

Search Procedure.

At query time, DiskANN performs a greedy best-first traversal. Starting from s, it maintains a candidate set L and visited set V. At each step, it selects

$$p^* = \arg\min_{p \in L \setminus V} d(p, q),$$

expands its neighbors, and updates the candidate set. The process continues until no better candidates remain, after which the top-k points are returned. To reduce SSD access overhead, DiskANN uses a beam search strategy, expanding multiple nodes (W) simultaneously instead of one, thereby reducing the number of I/O round trips.

Memory and Storage Design.

A key innovation of DiskANN is its hybrid memory layout. The graph structure and full-precision vectors are stored on SSD, while compressed vectors are kept in RAM. Compression is performed using Product Quantization (PQ):

$$\tilde{x}_p = PQ(x_p), \quad d(\tilde{x}_p, q) \approx d(x_p, q),$$

allowing fast approximate distance computations during search. When nodes are fetched from SSD, their full-precision vectors are retrieved alongside their neighbors, enabling accurate re-ranking without additional disk reads.

End-to-End Workflow.

The overall pipeline consists of two stages. During index construction, the dataset is used to build the Vamana graph, which is then serialized to disk. During query processing, the system starts from the medoid node, performs greedy beam search guided by compressed distances, retrieves candidate nodes from SSD, and finally re-ranks them using full-precision vectors to return the top-k neighbors.

Complexity and Insight.

Due to the multiplicative distance reduction property introduced by $\alpha > 1$, the number of graph hops required for convergence scales approximately as

$$O(\log n),$$

making DiskANN highly efficient even for very large datasets. Its performance stems from the combination of structured graph traversal, bounded-degree pruning, vector compression, and SSD-aware search optimization.

Limitations of DiskANN:

- **Static Graph Structure:**
  The Vamana graph is built only once and does not change over time, making it incapable of adapting to dynamic query distribution trends.
- **Distance-Based Only:**
  Edge addition relies solely on distance-based criteria, ignoring graph structure and traversal efficiency considerations.
- **Heuristic α-RNG Pruning Strategy:**
  The robust α-RNG pruning method is heuristic in nature and does not explicitly optimize any objective function.
- **Query Independence:**
  The algorithm does not incorporate feedback from queries, meaning the graph does not evolve based on real-world usage patterns.

Proposed Algorithmic Enhancements:

Adaptive Graph Evolution

We propose the development of a dynamic graph structure that evolves continuously based on search traversals. Frequently used edges are reinforced, while redundant or rarely used edges are pruned over time.

Entropy-Based Edge Pruning

Instead of heuristic α-RNG pruning, we introduce an entropy-based pruning strategy. This method selects neighbors that maximize information gain, ensuring more informative and diverse connections.

Traversal-Based Edge Scoring

Edge selection is guided by traversal efficiency rather than just proximity. Nodes are connected in a way that minimizes the number of hops required during search, improving overall navigation efficiency.

Experimental Design

To validate the proposed improvements, experiments will be conducted on the **SIFT1M** dataset. The performance of the enhanced algorithm will be compared against the baseline DiskANN implementation using the following metrics:
Recall@K, Average latency per query , Number of distance computations, Graph traversal hops

Conclusion

This paper analyzes the DiskANN framework and highlights its key limitations. We propose several enhancements based on adaptive, information-theoretic, and traversal-oriented approaches. These improvements aim to create a more efficient, dynamic, and intelligent approximate nearest neighbor search system.